```
module ActiveSupport
  class TestCase
    teardown do
      ActiveRecord::Base.subclasses.each(&:delete_all)
    end
  end
end
```

This is based on a Minitest version I got from Robert Evans on GitHub.[9]

The basic setup is similar. Load Minitest and ActiveRecord, and then load some files that will help in testing: minitest/autorun, which lets you run test files as standalone scripts; Mocha for mocks; active_support/test_case so you don't have to rewrite all your tests that use it; and minitest/reporters, which gives you color output at the terminal. You also have a couple of lines setting up the reporter.

By default, Minitest doesn't have the around behavior you used in RSpec. For the moment, you're doing a very simple and blunt thing, which is adding into your ActiveSupport::TestCase a teardown block that will be called after every test. It tells all known subclasses of ActiveRecord::Base to delete_all, cleaning up the database.

In practice this will likely be slower than the RSpec behavior, though for some reason SQLite seems to like it more. (If you want the around behavior in Minitest, you can get it with the minitest-around gem.[10]) And you can still use DatabaseCleaner if you want something fancier.

You then need to change the header of your test files as well. Here's a sample for project_test:

environment/01/test/models/project_test.rb

```
require_relative "../active_record_test_helper"
require_relative "../../app/models/project"
require_relative "../../app/models/task"
```

If you're trying to bypass Rails, don't use the Rails-provided rake testing tasks. All of them load the environment as a dependency, even if you don't explicitly do so in your test helper.

Prescription 50 The only way to know the boundaries of a new coding tool is to go past them. Try things.

9. https://gist.github.com/revans/4196367
10. https://github.com/splattael/minitest-around